Binary Classification of AOI Alarms into True Defects and False Calls

Second check of AOI results with supervised machine learning (Siemens production data)

P2: core code and baseline simulation · test plan · report

Yu Han · Melisa Subasi · Meron Hagos · Alejandro Rivas

Team 1 · Guided by Prof. Dr.-Ing. Hamidreza Heidari · 3 November 2026 · github.com/bodhi584/aoi-false-call-classification

Yu Han

Where the project stands

P1	20 Oct	Proposal, tech stack, MVP (revised topic 23 Oct)	**done**
P2	3 Nov	Core code + baseline simulation; test plan; report	**this phase**
P3	24 Nov	Feature progress; unit tests and CI; issues board	**next**
P4	8 Dec	Feature progress; full evaluation	**planned**
P5	22 Dec	Demo video, run instructions, packaging	**planned**
Final	12 Jan	Final presentation and report	**planned**

P2 in short

Literature review, 36 sources: on the same Siemens data a second check fails on later production days; the authors name two causes, drift and the threshold method.

P1 revised: keep their check as baseline and simulate its operation day by day, with retraining and threshold setting as the variables; compare with their later-slice results.

Core code: one-to-one replication of that study as the simulation baseline; first runs reproduce it.

Test plan written; report in docs/literature_report.pdf.

TECHNISCHE HOCHSCHULE DEGGENDORF THD

Yu Han

Revision of the P1 proposal after finding the core paper [8]

	What [8] already did	P1 plan (23 Oct)	Revised plan (P2)
Second check	classifier after AOI on the Siemens data; on a same-period test set ≤ 1 % missed defects	build our own second check	use the check of [8] as baseline (one-to-one)
Over time	evaluated on 5 later slices with a static model: 5–15 % missed defects	day-by-day simulation in time order	day-by-day simulation of operation on top of [8]: the part they did not do
Causes	names drift and the threshold method; tests neither	not known yet	address both: retraining against drift; a correctly set threshold
Models	random forest, balanced random forest (also kNN, XGBoost, AutoML)	logistic regression, random forest, neural network	same as [8], so results are comparable; model choice matters little [13]
Comparison	targets ≤ 1 % missed, ≥ 40 % false calls released	recall ≥ 99 %, specificity ≥ 20 % (board level)	same metrics as [8]; compared with their later-slice results

Unchanged: Siemens AOI data [5] · input → system → output · tools (Python, scikit-learn) · milestones

TECHNISCHE HOCHSCHULE DEGGENDORF

Melisa Subasi

AOI false calls are a real and current problem

≈ 99 %

of the solder joints flagged by AOI are false calls

Each flagged board is checked again by hand. Source: Pfab & Rothering (2025); data: Pfab et al. (2024), one Siemens SMT line, 132 days

Zero-defect manufacturing needs detection [1]

AOI checks every solder joint with a camera and fixed limits [2]; machine learning at the inspection step is an active field [3].

Manual workload [5]

Every flagged board goes to the manual inspection station; many are good boards.

Siemens and NVIDIA, Erlangen, Oct 2026 [4]

A second inspection step decides: real defect or false call. Target: 40 % fewer false calls (a target, not a result).

TECHNISCHE HOCHSCHULE DEGGENDORF THD

Meron Hagos

A second check with machine learning works on a test set

Study	Data	Classifiers	Result
Thielen et al. 2020 [6]	not public	kNN, neural network, random forest	false calls recognised; RF and NN best on large data
Albayari et al. 2026 [7]	Siemens AG	XGBoost, RF, DNN, kNN	XGBoost best balance of recall and precision
Pfab & Rothering 2025 [8]	Siemens, public	kNN, RF, balanced RF, XGBoost, AutoML	RF and balanced RF meet the targets on the test set

Which classifiers? [9]–[14]

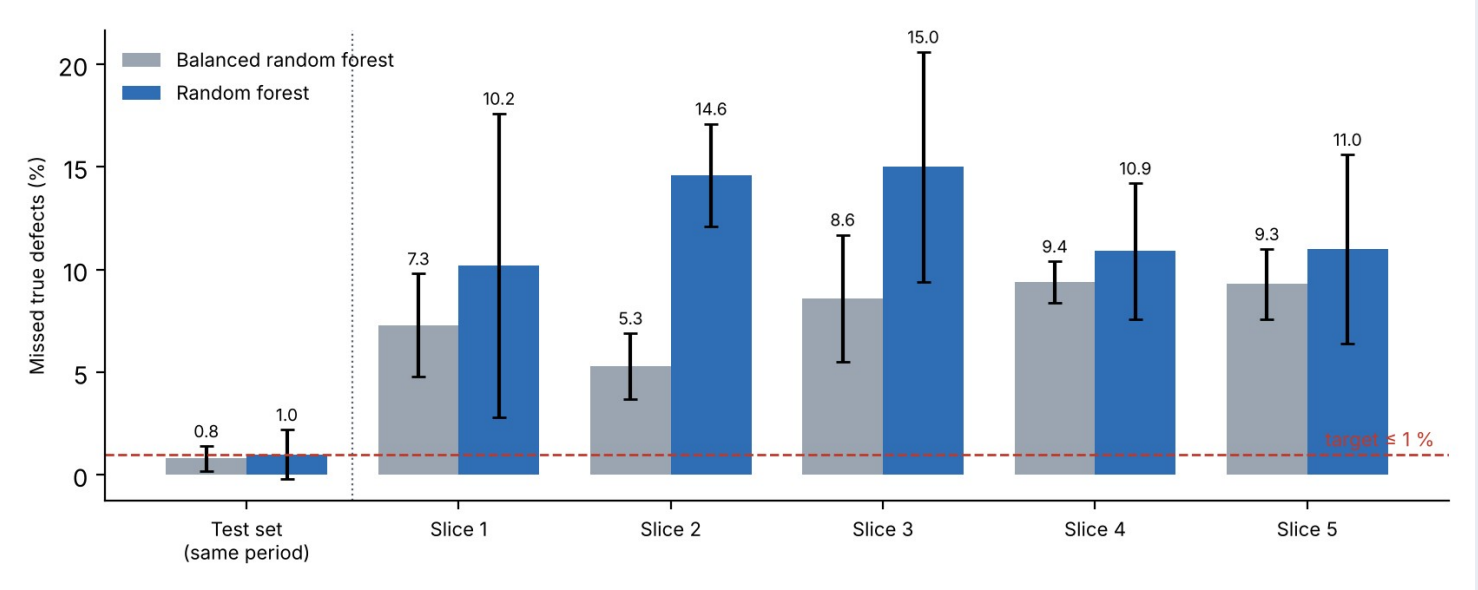
On tabular data, tree ensembles (random forest, gradient boosting) are the strongest family in five benchmark studies.

But: tested on data from the same period

None of these studies keeps the model running on later production days, except [8].

Alejandro Rivas

… but fails on later production days: drift



Why? Drift [21]–[27]

Pfab & Rothering: on target on the test set, 5–15 % missed defects on all five later slices.

Drift = the data change over time; a model trained on old data no longer fits [21]–[23].

Same effect in manufacturing: welding, semiconductor etch, sensors [24]–[26].

The paper also names an unstable threshold method.

Yu Han

Chain of evidence

Fails over time

train once → 5–15 % missed defects later [8]

Two causes

drift [21]–[27] and an unstable threshold method [8], [28]

Methods exist

threshold: [28]–[30]; retraining: [22], [27], [31]–[33]

Works elsewhere

fraud detection, <1 % positives [35]; semiconductor materials line [36]

Open for AOI

named by [8] as next steps, not tested for the second check of AOI results

Pfab & Rothering (2025), Sec. VII: "… potential next steps to stabilize the modeling by improving the method for setting the threshold and handling the distribution drifts within the dataset."

Our simulation focuses on exactly these two steps: the threshold method and drift (retraining).

Yu Han

Input → simulation → output

INPUT

Recorded Siemens production

AOI measurements of each flagged solder joint, in time order [5]

published models and settings of [8]

VARIABLES

What operation would change

Retraining: off / on, frequency (per slice → weekly → daily)

Threshold: as in [8] / set correctly / set on recent days

OUTPUT

Per production period

Missed true defects (%)

False calls released (%) = manual checks saved

next to the result of [8]

Reference lines from [8]: ≤ 1 % missed true defects, ≥ 40 % false calls released.

Later (P3–P5): a software with these variables as settings and charts of both outputs.

Yu Han

The two next steps of [8] become the variables of our simulation

Variable	Option	What it does	Sources
Threshold	as in [8]	mean of the five cross-validation thresholds	[8]
	order statistic with confidence	1 % bound held with 95 % confidence; needs ≥ 299 left-out true defects	[28]
	re-set for the current period	follows a changed share of true defects, also without labels	[29], [30]
Retraining	off	train once, as in [8]	[8]
	fixed schedule	per period, weekly, daily; growing or sliding window	[22], [31], [35], [36]
	triggered by drift	retrain when the error rises, or adapt the window	[32], [33]

Every option comes from a published method; the simulation shows its effect on missed defects and released false calls next to [8].

Yu Han

Replication of Pfab & Rothering (2025): one factor at a time

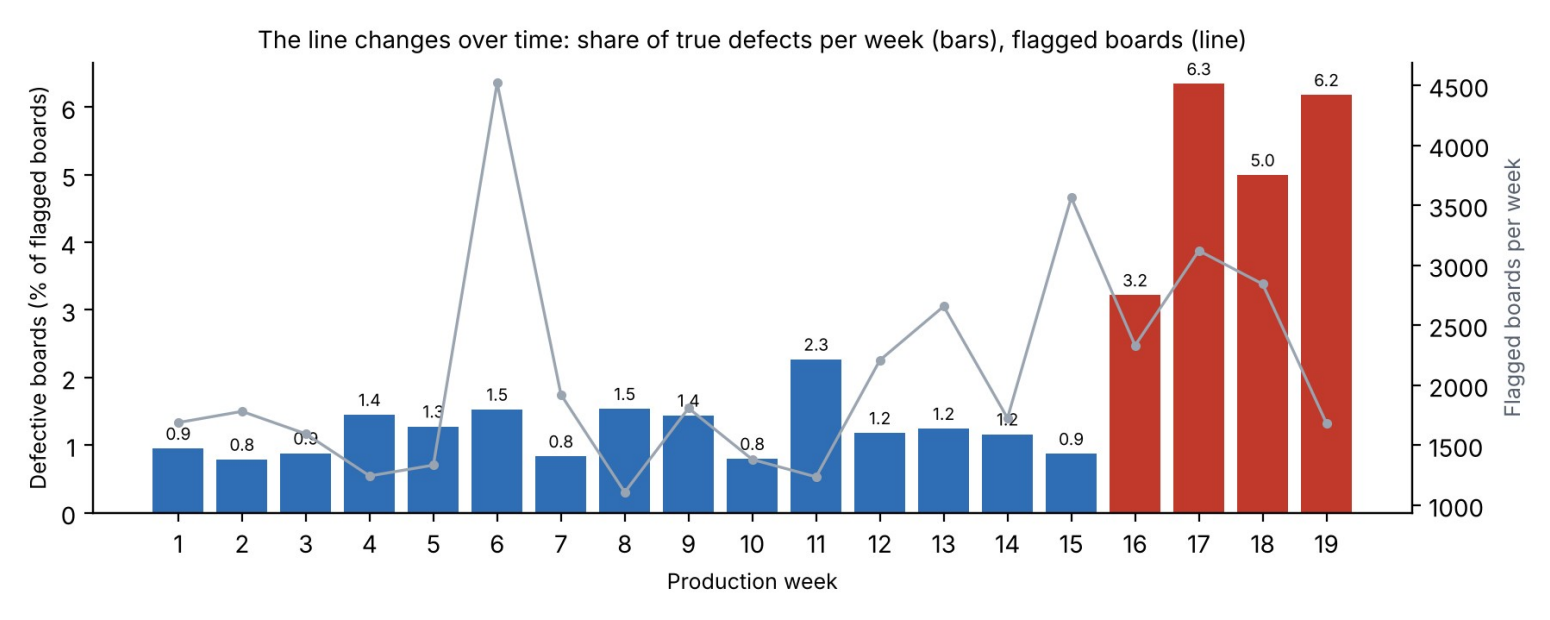
	(a) Paper exactly	(b) Train once, threshold re-estimated	(c) Retrain
Data split	same as [8]	same	same
Models, hyper-parameters, seeds	RF, balanced RF; logged values; 10 seeds	same	same
Threshold	value logged in [8]	their method, applied to the final model	same method, new data
Training	once	once	before each of the 5 later slices
Purpose	reproduce [8]	(a) vs (b) = effect of the threshold	(b) vs (c) = effect of retraining

Metrics on joint level, as in [8]: missed true defects (slip = 1 − recall) and false calls released (volume reduction = specificity). Assumption: manual-inspection labels are available for all joints (upper bound for retraining).

TECHNISCHE HOCHSCHULE DEGGENDORF

Alejandro Rivas

The line changes over time



Facts (our analysis)

440,274 flagged joints, 39,742 boards, 132 days

97.8 % of flagged boards are false calls

Share of defective boards: 0.8–2.3 % per week, then 3.2–6.3 % in the last 4 weeks

Drift test: early vs. late days separable, AUROC 0.974

Yu Han

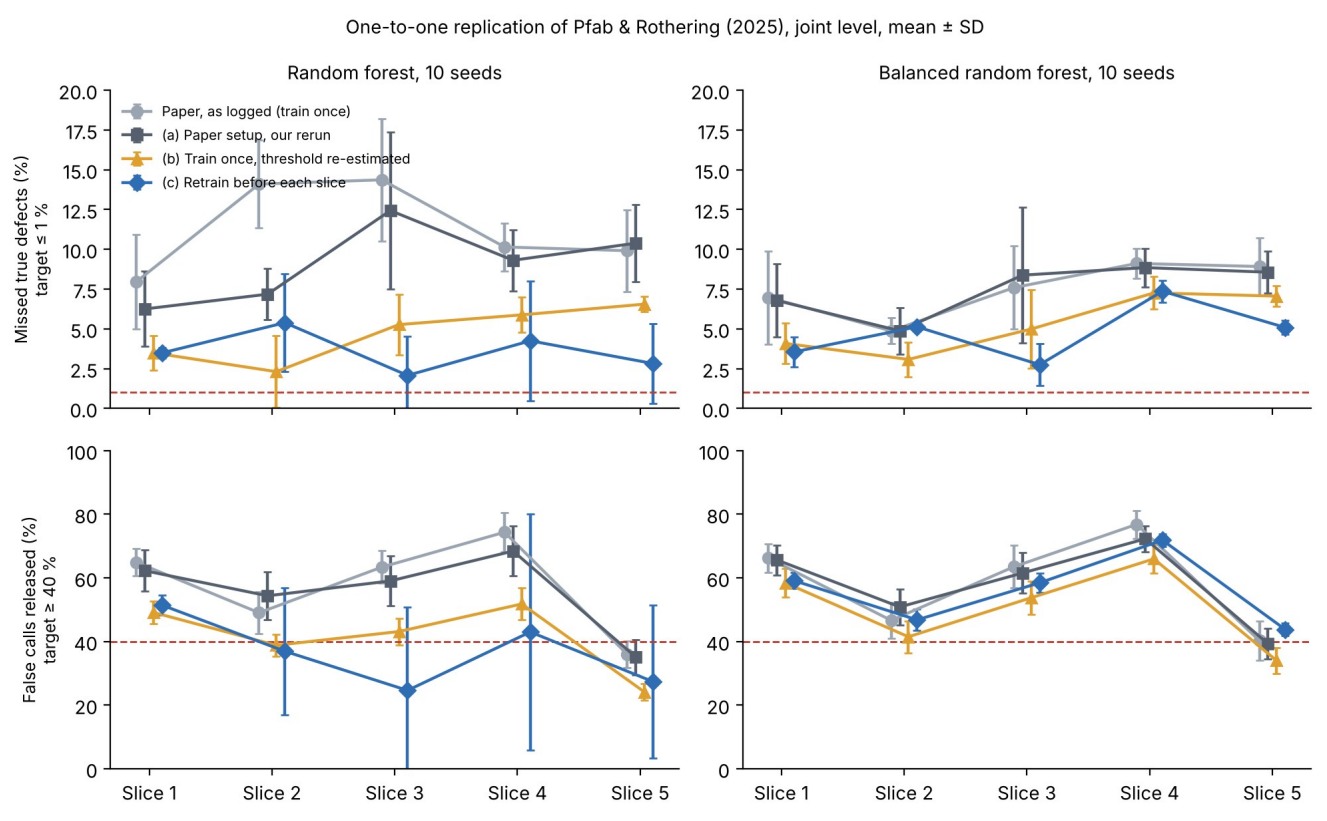
What runs in the repository

Part	File	What it does
Data	aoi_classification/data.py	load the Siemens data, boards, time order
Classifier + threshold	aoi_classification/model.py	train, score, threshold for 99 % recall
Baseline simulation	aoi_classification/simulation.py	day by day: train on earlier days only
Replication	scripts/replicate_pfab2025.py	one-to-one repeat of [8], train once vs. retrain
Classifier screening	scripts/compare_classifiers.py	13 classifiers, 8 families, same split
Tests + CI	tests/, GitHub Actions	10 tests on Windows, macOS, Linux

Tools: Python, pandas, scikit-learn, imbalanced-learn, matplotlib, Jupyter · Data: Mendeley (CC BY-NC 3.0), not stored in the repo

Yu Han

Baseline reproduced; threshold and retraining both lower missed defects



Reproduction ✔

Balanced RF: 7.5 % missed defects, as in [8].

Retraining: small gain

Balanced RF: 5.3 → 4.8 % missed, 50.7 → 56.0 % released.

RF: 4.7 → 3.6 % missed, but fewer released.

Threshold matters most

Correct threshold alone: 7.5–9.1 → 4.7–5.3 % missed.

Reference line ≤ 1 % not reached yet.

Melisa Subasi

How we check the code and the numbers

Group	Examples	Status
Data (D1–D7)	labels 0/1, no missing values, time order, checksum	4 exist, rest P3
Classifier + threshold (C1–C7)	threshold rule, all models run, same seed = same result	4 exist, rest P3
Simulation (S1–S4)	model only knows earlier days	1 exists, rest P3
Replication (R1–R4)	same split as [8]; (a) reproduces [8]; retraining uses only earlier data; 10 seeds	new
KPIs (K1–K5)	missed defects ≤ 1 %, false calls released ≥ 40 %; every number from results/	open

Unit tests run on every push (CI, three operating systems). A KPI that is not reached is a result, not a failed test.

Meron Hagos

Towards P3 (24 Nov)

Simulation software

one screen: retraining and threshold as settings; charts of missed defects and false calls released next to [8]

Retraining frequency

daily, every 3 days, weekly, per slice: how far do missed defects drop? [36]

Threshold with confidence

order statistic with enough left-out true defects [28]; re-set for the current period [29], [30]

More model families

gradient boosting, neural network, logistic regression in the same setup

KEY REFERENCES

Full list (36 sources) in docs/literature_report.pdf

[5] Pfab, K. et al. (2024). Data of automated optical inspection of surface-mounted technology electronic production. Data in Brief 53, 110110.

[8] Pfab, K., Rothering, M. (2025). Towards improved research methodologies for industrial AI: a case study of false call reduction. arXiv:2506.14521.

[21] Moreno-Torres, J. G. et al. (2012). A unifying view on dataset shift in classification. Pattern Recognition 45, 521–530.

[22] Gama, J. et al. (2014). A survey on concept drift adaptation. ACM Computing Surveys 46(4), 44.

[28] Tong, X., Feng, Y., Li, J. J. (2018). Neyman-Pearson classification algorithms and NP receiver operating characteristics. Science Advances 4(2), eaao1659.

[32] Gama, J. et al. (2004). Learning with drift detection. SBIA 2004, LNCS 3171, 286–295.

[35] Dal Pozzolo, A. et al. (2018). Credit card fraud detection: a realistic modeling and a novel learning strategy. IEEE TNNLS 29(8), 3784–3797.

[36] Gao, M. et al. (2026). Robust and reliable AI for predictive quality in semiconductor materials manufacturing with MLOps and uncertainty quantification. arXiv:2605.07752.

TECHNISCHE HOCHSCHULE DEGGENDORF

Repository

github.com/bodhi584/aoi-false-call-classification

Thank you

Team 1 · Yu Han · Melisa Subasi · Meron Hagos · Alejandro Rivas